CommerceFlow — Checkout Reliability Design Supplement v1.2

Status: TECHNICAL PROPOSAL — requires Tech Lead review before implementation

Baseline: Approved Checkout Policy v1.0; 10 business tables remain unchanged.

Scope: Checkout retries, crash recovery, inventory concurrency, cancellation, and test contracts.

1. Policy versus technical implementation

Approved: customer-scoped idempotency; minimum 24-hour completed-key retention; identical completed retries replay the original result; changed request returns HTTP 409; concurrent same-key request returns HTTP 409; only CONFIRMED orders may be cancelled; stock restoration occurs once.

Proposed here: concrete infrastructure schema, canonical fingerprint rules, transaction and recovery model, and retry responses. These details are not yet approved.

2. Infrastructure table: checkout_idempotency

This table is an infrastructure table, not an eleventh business entity. The migration must be reviewed with Flyway.

  | Column | PostgreSQL type | Constraint / purpose |

  | --- | --- | --- |

  | id | UUID | Primary key |

  | user_id | UUID | NOT NULL; references users(id) |

  | idempotency_key | VARCHAR(128) | NOT NULL |

  | request_fingerprint | CHAR(64) | NOT NULL; SHA-256 of canonical immutable checkout snapshot |

  | request_snapshot | JSONB | NOT NULL; immutable validated address, product IDs, quantities, unit prices, currency |

  | state | VARCHAR(16) | NOT NULL; IN_PROGRESS, COMPLETED, RETRYABLE_FAILURE |

  | order_id | UUID | NULL; references orders(id) |

  | response_status | SMALLINT | NULL; successful terminal response code |

  | response_body | JSONB | NULL; replayable terminal response |

  | lease_expires_at | TIMESTAMPTZ | NOT NULL; recovery eligibility for abandoned processing |

  | completed_at | TIMESTAMPTZ | NULL |

  | expires_at | TIMESTAMPTZ | NULL; at least completed_at + 24 hours |

  | created_at | TIMESTAMPTZ | NOT NULL |

  | updated_at | TIMESTAMPTZ | NOT NULL |

Constraints: UNIQUE(user_id, idempotency_key); index on (state, lease_expires_at); index on expires_at for cleanup. Enforce valid state/terminal-field combinations with CHECK constraints after implementation decisions are finalized. Use the authenticated principal as user_id, never a client-provided user ID.

3. Canonical fingerprint and cart mutation

On the first request, validate and persist an immutable snapshot containing normalized shipping address, sorted product IDs and quantities, server-side unit prices, USD currency, and a versioned serialization format. Hash canonical UTF-8 bytes with SHA-256. Do not accept a client-supplied total, price, or payment result.

For an identical retry after cart clearing, retrieve the saved record before loading the current cart. Compare the explicit request fields (including shipping address) against the saved canonical request and replay the saved result. Do not recompute the original cart fingerprint from the current, mutable cart. A request with the same key and different explicit checkout fields must return 409. Because cart contents are implicit in the checkout API, the saved cart snapshot is authoritative for replay; a fresh checkout must use a new key. Document this limitation in the API.

4. Proposed transaction and crash-recovery model

1. Authenticate Customer and validate CSRF, key format, and explicit request fields.

2. Atomically reserve (user_id, key) with a short lease; a second live request with the same key receives 409 IDEMPOTENCY_IN_PROGRESS and Retry-After.

3. For an existing COMPLETED record, compare the explicit request fingerprint against the saved request and replay the persisted response. For mismatches return 409 IDEMPOTENCY_PAYLOAD_MISMATCH.

4. Simulate mock payment before inventory mutation; no external financial transaction occurs. The simulation must be deterministic per reserved request or its outcome must be durably recorded, so crash recovery cannot silently change a previous result.

5. In one PostgreSQL transaction, conditionally decrement stock for each product in stable product-ID order, insert order and immutable item/address snapshots, clear purchased cart items, and mark the idempotency record COMPLETED with the replayable result.

6. If stock validation fails, roll back business changes; persist a documented terminal conflict result or an explicitly retryable state. The final choice must be consistent across SRS, API, and tests.

7. If the worker crashes while IN_PROGRESS, a lease-expiry recovery worker must acquire the key exclusively and inspect the order/result record before resuming. Never assume an expired lease means no order was committed.

8. Delete only expired terminal records after the approved retention period; do not delete active leases. Document operational cleanup and monitoring.

Open decision: Choose whether transient mock-payment failures are terminal/replayable or retryable under the same key. Until this is settled, the API must not promise one behavior.

5. Inventory and cancellation concurrency

Checkout must use an atomic conditional decrement (`stock_quantity >= requested_quantity`) and verify affected row count. Admin stock changes must use compatible concurrency control to prevent lost updates. Lock ordering should be deterministic when multiple products are purchased.

Cancellation must atomically transition `CONFIRMED -> CANCELLED` with a conditional update and restore stock only if exactly one order row transitioned. Competing cancellation requests must not each restock. `SHIPPED`, `DELIVERED`, and already `CANCELLED` orders cannot be cancelled in MVP. Include audit metadata for actor, timestamp, and reason where feasible.

6. API contract amendments (proposed)

• POST /api/v1/orders: required `Idempotency-Key` (nonempty, max 128 characters), authenticated session and CSRF token.

• 201: newly created order. Exact identical completed replay: original response status/body, optionally `Idempotent-Replayed: true`.

• 409 `IDEMPOTENCY_IN_PROGRESS`: concurrent same-key request; include `Retry-After`.

• 409 `IDEMPOTENCY_PAYLOAD_MISMATCH`: key reused with changed explicit request fields.

• 409 `INSUFFICIENT_STOCK`: inventory cannot fulfill request; no partial order.

• 403: invalid or missing CSRF token; 401: unauthenticated; 400: malformed input.

• PATCH /api/v1/admin/orders/{id}/status with CANCELLED: 200 on valid transition; 409 on invalid/repeated transition.

• Error responses use RFC 9457 Problem Details; do not expose SQL or session secrets.

7. Test cases required before implementation can be considered complete

  | Test ID | Scenario | Required assertion |

  | --- | --- | --- |

  | TC-IDEMP-001 | Same key, same request after cart clear | Same order ID; stock decremented once |

  | TC-IDEMP-002 | Same key, different shipping address | 409; no additional order |

  | TC-IDEMP-003 | Concurrent same-key checkout | One processing owner; no duplicate order |

  | TC-IDEMP-004 | Worker crash before commit | Recovery does not create duplicate order |

  | TC-IDEMP-005 | Worker crash after commit before HTTP response | Retry replays committed order |

  | TC-IDEMP-006 | Expired completed record | Cleanup respects minimum 24-hour retention |

  | TC-INV-001 | Two customers, stock=1 | Exactly one success; stock >= 0 |

  | TC-INV-002 | Admin stock update during checkout | No lost update |

  | TC-CANCEL-001 | Two concurrent cancellations | Stock restored exactly once |

  | TC-CANCEL-002 | Cancel shipped order | 409; no stock change |

  | TC-PAY-001 | Mock payment failure | No order or inventory mutation |

  | TC-SEC-001 | Missing CSRF / unauthorized access | Request rejected without side effects |

8. Documentation synchronization checklist

• SRS FR-11 and NFR-03: add immutable replay, crash recovery, cancellation rules.

• Physical ERD specification: add infrastructure table and distinguish from 10 business tables.

• OpenAPI: add all idempotency response codes, headers, and problem schemas.

• Checkout sequence: show reserve, replay, mismatch, crash recovery, and cancellation as separate flows.

• Test Strategy and Traceability Matrix: add test IDs and mark as PLANNED, not PASSED.

• ADR-006: reference this supplement as proposed implementation detail.

9. Release gate

This supplement is not an implementation authorization. Close open decisions, update the canonical OpenAPI and diagrams, visually verify exports, and obtain Tech Lead approval before documentation is labeled FINAL. No backend code or migrations were changed.